AOSP 进阶丛书 · 第六册 · ANDROID 13 (API 33)

PMS 与包管理

开机扫描建立包宇宙、Computer 快照读写分离、会话化原子安装五步管线、权限三层闸门与 ROM 预装正确姿势——一切组件的出生证明办公室。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：一切组件的"出生证明办公室" 2
第一章 开机扫描：包宇宙的建立 2
第二章 Computer 快照：读写分离的包宇宙 6
第三章 一次安装：会话化流水线 7
第四章 权限：三层闸门 10
第五章 卸载、多用户与查询 13
第六章 实战：ROM 预装应用的正确姿势（BSP 视角） 13
第七章 设计决策（十一问） 14
第八章 深水区：存储配额、多 APK 与备份恢复 15
第九章 补全：APK 签名体系、Intent 匹配与安装后布局 15
附录 A 观测实操 17
附录 B 快问快答（16 条） 17
附录 C 自检题 18
附录 D 动手实验室（半天量） 18
教科书习题集（分级） 18
术语表（按拼音排序） 19
排障手册：PMS 与包管理问题定位 19

本册速览

开机扫描建包宇宙（两本账：内存表+packages.xml）
Computer 快照读写分离 · 会话化原子安装五步 · 失败码词典
权限三层闸门（声明/授予/AppOps） · ROM 预装四件套
深水区：installd 配额/split/备份 · 补全：签名 v1-v4/Intent 匹配
存储与加密衔接：vold/FBE/keystore2（书11）

前言：一切组件的"出生证明办公室"

前四册里每个组件的启动都要"问 PMS 要地址"（queryIntentActivities、resolveActivity…）。本册终于解剖这座图书馆本身：**开机时它如何把全机 APK 读成一张包宇宙大表；一次安装如何原子地改写这个世界；权限这座三层闸门如何放行或拦截；dex 优化为何在 Android 13 交给 ART 模块**。

体例不变：源码逐段注释 + 逐段精读。本树（Android 13）PMS 有两处与旧教材显著不同的架构演进——**Computer 快照（读写分离）**与**安装/dexopt 职责外迁（InstallPackageHelper/ART 模块）**，本册会反复对照"旧世界"以防读者拿着旧地图找新路。

0.1 新旧世界对照表（读旧教材/旧博客前必看）

概念	旧世界（≤11）	本树（13）
安装执行	VerificationParams/ASEC	InstallPackageHelper（聚合器）+ InstallParams.installStage
查询	PMS 大锁内直接读	Computer 不可变快照（读无锁）
dexopt	installd 直接跑 dex2oat	ART 模块 artd 执行，PMS 只管策略
ActivityStackSupervisor	——	ActivityTaskSupervisor（书4 提及的更名）
解析	PackageParser	ParsingPackageUtils + AndroidPackage（pkg.parsing/）
root	隐式 root 用户	显式（多用户语义收紧）

逐段精读：读任何 2019-2022 年的 PMS 文章，先拿这张表换算名词，再看本树源码——**名词对不上的地方不是你错了，是架构走了**。

第一章 开机扫描：包宇宙的建立



图 1 PMS 启动扫描：入口、分区扫描顺序、两本账（内存表+packages.xml）

1.1 入口与两本账（逐段注释）

java frameworks/.../pm/PackageManagerService.java:1411 起（骨架+讲解注释）

```
public static Pair<PackageManagerService, IPackageManager> main(Context context, ...) {
// 讲解注释①：PackageManagerService 构造器是"重活总汇"：
// Settings.readLPw（Settings.java:2938）读 packages.xml——
// 上次关机时的包注册表（版本/权限/用户态/签名指纹）。
// 讲解注释②：Computer 快照起步——13 的核心改造：
// 所有查询走不可变快照（computer(...)），写操作持锁后重建快照。
// 读多写少的包宇宙从此"读无锁"。
// 讲解注释③：InitAppsHelper 扫描全部分区（下节）；
// 扫描结果与 Settings 对账：删掉"盘上没了"的，标灰"升级未完成"的。
// 讲解注释④：writeLPw 落盘 + 对外发布（IPackageManager binder 就绪，
// SystemServer 的 Bootstrap 批才放行——书2 service-deps 图的 PMS 节点）。
}
```

逐段精读：①的"对账"解释了拔卡/损坏 APK 的自愈：盘上消失 → PackageSetting 删除；hash 不符 → 重装标记。**注册表（账）与文件（物）双轨，开机必对账**——数据库思维的系统级应用。

1.2 分区扫描：顺序即权限（InitAppsHelper 逐段注释）

java pm/InitAppsHelper.java:308-342 scanSystemDirs（骨架+讲解注释）

```
// 讲解注释①：scanSystemDirs(:308) 统管只读分区扫描，顺序即信任序：
// scanDirTracedLI(:325) framework 目录（framework-res 等）最先入库——
// 后续一切包依赖它的资源与共享库。
// 讲解注释②：循环各分区（system/vendor/product/system_ext）：
// :337 partition.getPrivAppFolder()（priv-app，特权由 privapp-permissions
```

```
// 白名单二次裁定）→ :342 partition.getAppFolder()（普通 app）。
// 讲解注释③：:320 partition.getOverlayFolder()（在 app 之前先收 RRO
// 资源覆盖层，供版本匹配）。
// 讲解注释④：只读分区扫完，:248 getAppInstallDir()（/data/app）最后对账：
// 用户装的可覆盖系统同包（升级语义），与 packages.xml 增量核对。
```

扫描顺序=优先级基线：framework → system(priv→normal) → vendor → data。同包名高版本胜出（升级），签名不符则拒绝覆盖——这是"系统 App 静默升级"的机制底座，也是恶意 App 冒名系统包的封印。

> **源码锚点 · 第一章锚点**
> PackageManagerService.java:1411 main / Settings.java:2938 readLPw / :175 类定义
InitAppsHelper.java:248/320/325 三处 scanDirTracedLI
解析器：pm/pkg/parsing/ParsingPackageUtils（APK→ParsingPackage→AndroidPackage）

1.3 单个 APK 的入库之旅（逐段注释）

java pm/InitAppsHelper→ParsingPackageUtils（骨架+讲解注释）

```
// 讲解注释①：scanDirTracedLI 遍历目录每个 .apk：
// ParsingPackageUtils.parsePackage → ApkLite（轻解析：只要 manifest 头）
// ——先看"值不值得全量解析"（版本/签名初判），省开机时间。
// 讲解注释②：全量解析 ParsingPackage：四大组件逐个读进内存对象
// （Activity/Service/Receiver/Provider + intent-filter 列表）；
// 权限声明（uses-permission/permission 自定义）一并入库。
// 讲解注释③：签名校验：apksigner 验签 → PackageSignatures 指纹记账
// （升级一致性与 platform 签名特权判定的依据）。
// 讲解注释④：建索引：组件按 intent-filter 写入倒排表
// （mActivities/mServices/mReceivers/mProviders + queryIntent 快速通道），
// 供第四册所有"问地址"消费。
// 讲解注释⑤：设置默认 user 状态（enabled），priv-app 特权白名单比对。
```

逐段精读：①的"轻解析先行"是开机性能的关键取舍——全量解析（资源表/代码段偏移）推迟到真正需要时；②与④解释了 queryIntentActivities 为什么是 O(filter 命中) 而非全表扫描。**包宇宙的本质=以 manifest 为 schema 的内存数据库**，扫描就是全量导入。

1.4 packages.xml 字段速查（Settings 账本）

字段	含义	谁写
name/codePath	包名与 apk 落点（v~~随机目录）	安装五步第 2 步
versionCode/tsTime	版本与更新时间	安装/升级
signatures/cert	签名指纹	扫描③
perms（granted/denied）	权限三态（per-user）	授予/撤销路径
pkgFlags/privateFlags	SYSTEM/PRIVILEGED 标记	扫描分区归属

字段	含义	谁写
enabled/components enabled	包与组件级启停	pm enable/disable
installSource	谁装的（initiator/installer 包名）	会话提交时
firstInstallTime/lastUpdateTime	审计三兄弟	安装五步

观测：`adb shell su 0 cat /data/system/packages.xml | head -50`（userdebug）。**看一次真实文件胜过读十遍字段表。**

1.5 PMS 扫描完整源码走读（教科书级逐步分解）

第①段：从 SystemServer 调用 PMS.main

java frameworks/base/services/java/com/android/server/SystemServer.java:1233（骨架+注释）

```
// 讲解注释①：PMS.main 在 Bootstrap 批被调用——
// 书2 §七的 Bootstrap 批列表中，AMS 之后、phase 100 之后
Pair<PackageManagerService, IPackageManager> pmsPair =
PackageManagerService.main(mSystemContext, installer,
domainVerification, mFactoryTest != FactoryTest.FACTORY_TEST_OFF,
mOnlyCore); // mOnlyCore：安全模式下只扫 core 包
// 讲解注释②：mOnlyCore 在安全模式（Safe Mode）时为 true——
// 只加载核心系统包，跳过所有普通 App——"安全模式最小集"
```

第②段：PMS 构造器逐段（PackageManagerService.java）

java pm/PackageManagerService.java 构造器（骨架+讲解注释）

```
public PackageManagerService(Context context, Installer installer,
boolean factoryTest, boolean onlyCore) {
// 讲解注释③：Settings.readLPw（Settings.java:2938）
// 读 packages.xml——上次的包注册表（版本/权限/签名指纹/路径）
// 对账：删掉"盘上没了"的包、标灰"升级未完成"的包
// 讲解注释④：mSettings.mResolvedPermissions → 权限树初始化
// 讲解注释⑤：scanDirTracedLI 扫描（InitAppsHelper :248/:320/:325/:337/:342）
// 顺序：framework → priv-app → app → vendor → product → data
// 讲解注释⑥：writeLPw 写回 packages.xml（先写临时文件→原子 rename）
}
```

第③段：扫描单个 APK 的全流程

text ParsingPackageUtils.parsePackage 的五步（读码归纳）

```
步骤 1：ApkLite 轻解析（读 ZIP 中央目录，只要 manifest 头）
→ 值不值得全量解析？（版本对比/签名初判）
步骤 2：全量解析 → ParsingPackage 对象
→ 读 AndroidManifest.xml 全部标签
→ 四大组件 + intent-filter + 权限声明 + 共享库
步骤 3：签名校验 → ApkSignatureVerifier（core/java/android/util/apk/）
→ v2/v3（SigningBlock）→ v1（META-INF 兜底）
步骤 4：建索引 → intent-filter 写入倒排表
→ mActivities/mServices/mReceivers/mProviders
→ queryIntent 的 O(filter命中) 查询基础
```

```
步骤 5：PackageSetting 创建 → Settings 立项
→ priv-app 特权白名单比对（privapp-permissions-*.xml）
```

第④段：scanDirTracedLI 逐行注释

java pm/InitAppsHelper.java:308-342（骨架+讲解注释）

```
private void scanSystemDirs(PackageParser2 packageParser, ...) {
// 讲解注释①：:325 framework 目录最先扫——
// framework-res.apk 等最先入库（后续包依赖它的资源/共享库）
scanDirTracedLI(frameworkDir, null, ..., packageParser, executorService);
// 讲解注释②：:320 overlay 目录在 app 之前——
// RRO 资源覆盖层先收（版本匹配优先级高于 app）
scanDirTracedLI(partition.getOverlayFolder(), ..., packageParser, ...);
// 讲解注释③：:337 各分区 priv-app → 特权包（白名单二次裁定）
scanDirTracedLI(partition.getPrivAppFolder(), ..., packageParser, ...);
// 讲解注释④：:342 各分区普通 app → 系统应用
scanDirTracedLI(partition.getAppFolder(), ..., packageParser, ...);
// 讲解注释⑤：:248 最后扫 /data/app —— 可覆盖系统同包
scanDirTracedLI(mPm.getAppInstallDir(), ..., packageParser, ...);
}
```

设计洞察 · 逐段精读：扫描顺序为什么是"先系统后数据"

优先级=信任级。system 分区的包在 PMS 眼里"更可信"——data 分区的同包名高版本 APK 可以覆盖 system 版本（升级语义），但签名必须一致（信任链不中断）。**信任边界=分区边界**，这是 PMS 扫描的核心设计原则。

第二章 Computer 快照：读写分离的包宇宙

Android 13 前的 PMS 是"一把大锁管所有查询"；13 起 Computer（pm/Computer.java）成为查询世界：每个写操作（安装/卸载/权限/禁用）在锁内完成**变更+重建快照**，读者永远拿一致的旧快照，写完原子换引用。收益：并发查询不再互相排队（AMS/WMS 每秒千次 queryIntent 不再与安装互卡）；代价：写路径多一次快照构建（安装本就低频）。

2.1 Computer 逐段注释（换引用的瞬间）

java pm/PackageManagerService/Computer（骨架+讲解注释）

```
// 讲解注释①：构造：把 mSettings+mPackages 等视图打包成不可变对象；
// mComputer 字段 volatile——读者永远看到"旧快照"或"新快照"之一，
// 没有"装了一半的包宇宙"（可见性与原子性的教科书应用）。
// 讲解注释②：查询入口族：queryIntentActivitiesInternal/
// checkUidPermission/getApplicationInfo... 全部只读快照，
// 不再 synchronized(PMS.this)——AMS 的千级 QPS 从此不与安装互锁。
// 讲解注释③：写路径：PMS 写方法在 mWriteLock 内改 mSettings/mPackages
// → new Computer(...) 重建 → 赋回 mComputer（原子换引用）→ 放锁。
// 安装低频，重建快照的毫秒级成本被写频率摊薄。
```

逐段精读：③的"改-建-换"三步让读写并发度拉满，代价是**同一次安装内的多次读可能看到旧快照**——但安装的可见性本来就该以"广播+完成回调"为界，这正是语义与实现的精确对齐。

先修要求与学习路线

先修：已读前三册（架构/启动/Binder）。

读完本书你将能够：

- 追踪PMS 与包管理的核心机制的完整调用链
- 解释关键设计决策的"为什么"
- 使用 dumpsys 定位本领域问题
- 回答相关面试题（书7 对应章节）

> **设计洞察 · 逐段精读：为什么 PMS 最需要这个改造**
> PMS 是全机读频率最高的服务（每次 startActivity/Service/bind/sendBroadcast 都要 queryIntent），而写极稀疏（装/卸）。读写比万比一的场景，读写分离是教科书式解法——与 Binder 的"注册表查询缓存"（书3 §7.1）、system_server 的 LocalServices 同属一族：**把"热读冷写"的服务从锁里解放出来**。

第三章 一次安装：会话化流水线



图 2 adb install 全管线：会话准备→校验落盘→编译记账→广播生效，含回滚与观测

3.1 PackageInstallerSession 逐段注释

java pm/PackageInstallerSession.java:186/1638（骨架+讲解注释）

```
public class PackageInstallerSession extends IPackageInstallerSession.Stub {
```

```
// 讲解注释①：会话三态：数据写入（staging 目录 /data/app/v~~随机/）
// → commit(:1638）才进入管线；此前随时可放弃，零副作用。
// 讲解注释②：v~~ 目录随机化：防"预放旧版再升级"的路径猜测攻击。
// 讲解注释③：流式写入：Play 增量下载/ADB stream 都写同一会话，
// 支持多 APK（base+split）一次提交。
// 讲解注释④：commit → InstallParams.installStage(:320/333)
// → InstallPackageHelper（13 的安装聚合器，旧教材的 VerificationParams/
// ASECInstallArgs 等已在此收编）。
}
```

3.2 落盘五步与广播

校验（签名/版本/sharedUserId 同源）→ FileInstallArgs 拷贝到正式目录 → manifest 解析入 mPackages（Computer 换新快照）→ 旧版 APK 入删除队列 → Settings.writeLPw（**临时文件+原子 rename**，掉电不烂账）。生效信号是**有序广播链**：`PACKAGE_REPLACED → PACKAGE_ADDED(REPLACING) → ...`，MA（Managed Profile）与备份服务等按序感知。全链任何一步失败：staging 丢弃，旧版全程在线——**用户永远感知不到"装了一半"**。

3.3 dexopt：为什么搬去 ART 模块

13 起 dex 编译的执行者从 installd 移交 **artd**（com.android.runtime APEX 内，Mainline 可升级）：PMS/ArtManagerService 只做策略（何时/何种 profile），ardt 负责执行 dex2oat（书1 §8.1 三级执行的落点）。动机：**编译器与运行时必须同版本进化**（profile 格式/ABI 变更），分家即版本漂移。installd 退守存储管家：目录属主、配额（CacheTracker）、crate 管理。

3.4 广播链与回滚（12+）

安装收尾的有序广播不是"通知一下"那么简单，链条的每一环都有系统行为挂钩：

text PACKAGE_REPLACED 广播链（有序，按注册序）

```
1 PMS 自己的接收器：更新运行时权限态（升级保留授予）、备份记账
2 备份服务：新版本纳入备份集
3 Managed Profile/设备策略：工作资料内同步判定
4 备份/输入法等系统组件：默认应用重指向（若被替换的是键盘等）
5 最后才是第三方接收者（你 App 里的 PACKAGE_ADDED 监听）
```

回滚（Rollback，Android 12+）：升级时 `RollbackManager` 记录旧版 staging 与数据快照指针；新版崩溃率超阈值（Watchdog 判定）自动回退——安装管线的"后悔药"，本质是给会话多挂一个反向事务。`pm uninstall-staged/dumpsys rollback` 可观测。

3.5 安装失败码全表（排障词典）

失败码	含义	常见诱因
INSTALL_FAILED_ALREADY_EXISTS	同名已装且非覆盖	-r 缺失
INSTALL_FAILED_INVALID_APK	解析失败	损坏/manifest 非法

失败码	含义	常见诱因
INSTALL_FAILED_INVALID_URI	路径不可达	URI 权限
INSTALL_FAILED_INSUFFICIENT_STORAGE	空间不足	/data 满
INSTALL_FAILED_DUPLICATE_PACKAGE	同名同版本已存在	重复提交
INSTALL_FAILED_NO_SHARED_USER	sharedUserId 协商失败	目标 uid 冲突
INSTALL_FAILED_UPDATE_INCOMPATIBLE	签名不一致	覆盖安装换了签名
INSTALL_FAILED_MISSING_SHARED_LIBRARY	依赖库缺失	uses-library 未声明
INSTALL_FAILED_CONTAINER_ERROR	存储容器错	vold/卷异常
INSTALL_FAILED_VERIFICATION_FAILURE	校验拒绝	验证器/策略拦截

逐段精读：这些码不是异常文案，是**管线每一步的"门牌号"**——拿到码先回 §3.2 对应哪一步，再下钻那一门（解析/校验/存储/签名），排障路径立即收窄到单点。

3.6 安装管线完整源码走读（教科书级逐步分解）

第①段：PackageInstallerSession 准备

java pm/PackageInstallerSession.java:186/1638（骨架+讲解注释）

```
public class PackageInstallerSession extends IPackageInstallerSession.Stub {
// 讲解注释①：每个 install 命令对应一个 Session（有 ID）
// 会话三态：数据写入→commit→完成/失败
// 数据写入（staging 目录）：/data/app/v~~<随机32位>/staging
// 随机目录防"预放旧版再升级"的路径猜测攻击
public void commit(@NonNull IntentSender statusReceiver, boolean forTransfer) {
// :1638
// 讲解注释②：commit 触发校验→copy→dexopt→记账→广播 五步
// 此前一切可放弃，零副作用（事务性）
// 讲解注释③：InstallParams.installStage(:320)
// → InstallPackageHelper（13 的安装聚合器）
}
}
```

第②段：校验（签名/版本/共享库）

text InstallPackageHelper 的校验链（读码归纳）

```
① 签名一致：新旧版本签名必须匹配（v2/v3 指纹对比）
→ 不一致 → INSTALL_FAILED_UPDATE_INCOMPATIBLE
② 版本码：新 >= 旧（降级拒绝）
→ 覆盖安装时校验
```

```
③ 共享库：uses-library 声明的库是否可用
→ 缺失 → INSTALL_FAILED_MISSING_SHARED_LIBRARY
④ 权限冲突：sharedUserId 相同则签名必须一致
```

第③段：落盘（copy + dexopt）

text 安装五步的物理操作

```
① copy：staging → 正式目录 /data/app/v~~<hash>/c~~<hash>/
（APK 的最终位置，包含 base.apk 和 split_*.apk）
② 解压：lib/<abi>/*.so（extractNativeLibs=false 时不解压）
③ dexopt：13 起移交 ART 模块的 artd（书6 §三.3）
→ speed-profile 编译（只编热点方法）
④ 设置目录属主：chown 到目标 uid → vold 补 SELinux 标签
⑤ Settings.writeLPw：packages.xml 落盘（先写临时→rename 原子）
```

第④段：广播链

text 安装完成的有序广播链（按优先级）

```
1. PMS 自己：更新运行时权限态、备份记账
2. 备份服务：新版本纳入备份集
3. 工作资料/设备策略：同步判定
4. 系统组件：默认应用重指向（若替换的是键盘）
5. 最后才是第三方接收者（你 App 里的 PACKAGE_ADDED）
```

第⑤段：失败回滚

text 任何一步失败 → staging 目录丢弃 → 旧版全程在线

```
// 讲解注释：用户永远感知不到"装了一半"——
// 这是 A/B 安装模型的原子性保证
// 对比老世界（Android 5 前）：install 中断 → 系统坏 → 必须 re-flash
```

设计洞察 · 逐段精读：为什么要会话化

三个维度：**原子性**（staging 隔离，任意失败=回到旧版）、**流式**（APK 边下载边写，Play 增量/免完整包）、**可审计**（InstallSource 记录"谁装的"）。**会话化=把安装从"命令"变成"事务"**——这是包管理从粗放到精炼的标志。

第四章 权限：三层闸门



图 3 权限检查调用栈与运行时权限三态机、默认授予、观测

4.1 检查栈逐段注释

text 权限检查栈（对照 pms-perm 图）

```
context.checkPermission(PERM, pid, uid) ← App/服务侧入口
→ PMS(Computer).checkUidPermission ← 快照查询
→ PermissionManagerService.checkUidPermission(:226→:238)
→ 三层判定：
① 安装时声明过吗？（manifest requestedPermissions）
② 运行时授予态？（per-user 三态机）
③ AppOps 即时开关？（用户"仅这次"/撤销）
→ 特例通道：platform 签名、sharedUserId、root/system uid 白名单
```

逐段精读：③是常被忽略的一层——**"授予"不等于"每次都过"**（AppOps 可在使用时再拦），相机的"仅在使用期间允许"即三层联防。对照 SELinux（书1 第十一章）：权限管"应用语义"，SELinux 管"进程行为"，两墙独立不互代。

4.2 默认授予与特权白名单

预装应用的"出厂即有权限"由 `DefaultPermissionGrantPolicy` 按角色（拨号/短信/浏览器…）授予；priv-app 的特权权限必须同时出现在 `privapp-permissions-com.xxx.xml` 白名单——**特权=分区位置+白名单双条件**，缺一构建期即拒（`host_configverification`）。升级时既有授予态保留（`PACKAGE_REPLACED` 路径不清账）。

4.3 权限对话框的真实路径

`requestPermissions` 并不是 App 直接弹窗：`Activity.requestPermissions` → `PackageManager.requestPermissionAssets…` 最终进 **PermissionController**（独立 APK，Mainline

模块）——grantRequest 走回 PMS 的 grantRuntimePermission。**UI 与账本分离**：对话框属 Controller（可独立升级换设计），授予动作仍是 PMS 单点记账。这正是 Mainline 化的典型手法：**把"策略与界面"搬进模块，把"事实与账本"留在系统**。

特殊权限通道：SYSTEM_ALERT_WINDOW 走设置特批页（ACTION_MANAGE_OVERLAY_PERMISSION）；ACCESS_BACKGROUND_LOCATION 强制二段授权（先前台后后台）；BIND_DEVICE_ADMIN 等签名级权限连弹窗资格都没有——**对话框本身就是一层权限分级的 UI 体现**。

4.4 权限三层闸门完整代码走读（教科书级逐步分解）

第一层：安装时声明（静态检查）

xml App 的 AndroidManifest.xml

```xml
<uses-permission android:name="android.permission.CAMERA" />
```

java PMS 侧的处理（pm/PermissionManagerService.java 汇总）

```
// 讲解注释①：uses-permission 声明被 ParsingPackage 解析入库
// → requestedPermissions[] 数组（属于 AndroidPackage 对象）
// → 每次系统启动时重新比对：新权限默认 deny（运行时权限）
// → 新增的签名级权限需检查签名是否与声明方匹配
```

第二层：运行时授予（用户意志）

java PermissionManagerService.java:226 → :238

```
private int checkUidPermission(int uid, String permName) {
// 讲解注释②：入口方法——所有权限检查的汇聚点
// 先查权限委托（Delegate），再走默认实现
return mPermissionManagerServiceImpl.checkUidPermission(uid, permName);
}
// PermissionManagerServiceImpl.checkUidPermission 的内部逻辑：
// 讲解注释③：三层判定按序执行：
// a）安装时声明过吗？（requestedPermissions 包含此权限？）
// b）运行时授予态？（PackageUserState 中 granted=true？）
// c）AppOps 即时开关？（AppOpsManager.checkOp()）
// → 三层全过 → PERMISSION_GRANTED / 否则 → PERMISSION_DENIED
```

第三层：AppOps 即时控制

java AppOpsManager 的角色（讲解注释）

```
// 讲解注释④：AppOps 是"使用审计+即时开关"——
// b）记录每次使用的次数/时长/时间戳
// c）运行时可以动态拦截（如"仅使用期间允许"= 超时自动回 deny）
// d）与运行时权限的关系：权限是"门钥匙"，AppOps 是"门卫日志"
```

权限三层闸门完整链

text App 调用 camera.open() 时的权限检查链

```
1. App 代码调 camera.open()
2. CameraManager → CameraService（书12 §二）
3. CameraService.enforceCallingPermission（书3 §五・身份切换）
```

```
→ Context.checkPermission(CAMERA, pid, uid)
4. → PMS.checkUidPermission (:226→:238)
→ 三层判定（声明/授予/AppOps）
5. → 通过 → CameraService 返回 CameraDevice
→ 失败 → SecurityException 或 CameraAccessException
// 讲解注释⑤：SELinux 在更底层管"进程能不能连 camera_service"
// 权限层管"App 语义上许不许可"——两道独立的墙
```

设计洞察 · 逐段精读：为什么权限要三层而不是一层

变更频率不同：声明极少变（静态）、授予偶尔变（用户操作）、AppOps 高频变（使用中动态调整）。**把变更频率不同的策略分层存储**，每层可独立缓存和撤销——这是"按变更频率分治"的设计直觉。

第五章 卸载、多用户与查询

- 卸载：`deletePackage` → FileInstallArgs 清目录 → Settings 删账 → `PACKAGE_REMOVED` 有序链（含 `EXTRA_REMOVED_FOR_ALL_USERS` 语义）。
- 多用户：包本体一份，`PackageUserState` per-user 记账（enabled/distraction/授权）——`pm install --user` 的机制底座；工作资料（Managed Profile）再叠隔离。
- 查询的五个入口家族：queryIntentActivities/Services/Receivers/ContentProviders/resolveActivity——全部走 Computer 的**组件倒排索引**（intent filter 注册表），这是第四册每次 startActivity"问地址"的索引结构。

第六章 实战：ROM 预装应用的正确姿势（BSP 视角）

系统开发常任务"把一个 App 塞进 ROM 且给对权限"，标准路径四条：

- **放哪**：/system/app（普通系统应用）、/system/priv-app（特权，配 privapp-permissions 白名单）、/product/app（产品定制，OTA 不随 system 升级）、/data/app（伪预装，可卸载）。
- **怎么声明**：`Android.bp` 里 `android_app { name:..., privileged: true, ... }`（Soong 视角，书1 §7.2）；证书选 platform（等同系统签名）或共享证书。
- **默认权限**：`DefaultPermissionGrantPolicy` 的例外名单文件 `etc/default-permissions/*.xml`。
- **资源与配置定制**：RRO overlay（`device/<oem>/<product>/rro/`）覆盖框架资源——改默认值不改框架代码的官方姿势。

设计洞察 · 逐段精读：预装三问

能不能卸？（system 不可、product 视配置、data 可）；能不能静默升级？（同签名覆盖安装即可）；能不能要任意权限？（priv 白名单过不了构建期校验——**特权是协商出来的不是声明出来的**）。这三问几乎覆盖厂商接入的全部合规审查点。

5.1 多用户与工作资料的语义表

操作	效果	命令
install --user 0	只装给主用户	pm install --user 0
install --user 10	装给副用户（工作资料）	同上换 id
uninstall --user 10	仅该用户消失（本体保留）	pm uninstall --user 10
enable/disable --user	per-user 启停	pm disable-user --user 10
默认应用 per-user	拨号/键盘可各用户不同	RoleManager 按用户记账

工作资料（Managed Profile）= 副用户 + DevicePolicy 沙箱 + 独立锁屏挑战——它是全家桶隔离（通知/权限/VPN 全套 per-profile），不只是"另一个用户"。PMS 的 per-user 记账（PackageUserState）是这一切的地基：**包是一等公民，用户是包的属性**。

第七章 设计决策（十一问）

- **为什么扫描顺序固定且 system 在前？** 升级覆盖与签名信任的方向性；framework 资源是依赖根。
- **为什么账（Settings）与物（APK 文件）分离？** 掉电/损坏可对账自愈；安装原子性由"先物后账再广播"三段保证。
- **为什么 Computer 快照？** 热读冷写服务的读写分离，安装不再卡查询。
- **为什么会话化安装？** 原子性+流式+可审计（InstallSource 记录"谁装的"）。
- **为什么 dexopt 移交 ART？** 编译器与运行时同版本进化，Mainline 可独立升级。
- **为什么权限三层闸门？** 声明（静态）+授予（用户意志）+AppOps（即时控制）对应三种变更频率，各层独立缓存与撤销。
- **为什么权限 UI 独立成 PermissionController 模块？** 界面高频迭代、账本必须稳定——Mainline 分界线画在"策略可变/事实不变"处。
- **为什么 rollback 只对 staged 会话开放？** 回滚需要原子反向事务，普通安装没有 staging 目录做物质基础。
- **为什么 enabled 状态分 per-user 与组件两级？** 用户可见语义（禁用某 App）与系统语义（禁用某接收器省电）是两种生命周期，合并记账会互相误伤。
- **为什么静默安装是系统级权限？** 安装=写入代码执行权，等同系统信任边界外扩——必须有签名级把关。

第八章 深水区：存储配额、多 APK 与备份恢复

8.1 installd：配额与缓存清退官

`frameworks/native/cmds/installd/`：以 root 运行、binder 对接 PMS/系统（CacheTracker.cpp:36 起）。三件职责：**目录属主托管**（chown 到目标 uid/改 SELinux 标签——安装五步里 PMS 无权做的脏活）、**cache 配额与清退**（CacheTracker 按 app 聚合 cache 体积，磁盘紧张时由系统存储策略调用 rm_cache 级联清退，App 无感）、**Crate/私有目录**（scoped storage 的物理执行者）。配额底层是文件系统原生 quota 或 xattr 记账（按设备配置）——所以"cache 莫名变小"第一嫌疑就是它。

8.2 多 APK 与 Feature Delivery 的系统侧

split APK（base+config/split_*）在系统侧=一个 Package 多个 APK 文件（PackageApkInstaller 路径），签名须全一致；加载由 LoadedApk 的 split classloader 路由（衔接书 4 §1.4 注释①）。Play Feature Delivery 的 install-on-demand/conditional 只是**会话化安装**（书6 §3.1）在模块粒度上的复用——系统无需新机制，这是会话化设计的红利兑现。

8.3 备份恢复体系

三套通道并存：**Key/Value 备份**（BackupManager API → BMS（services/backup）→ 传输（Google/ OEM）逐键增量）；**全量备份**（adb backup / Auto Backup tar 流）；**设备到设备**（近场直传，13 主推）。关键机制：BMS 在安装会话后建"备份过期"标记，应用首次启动懒注册 Agent；恢复发生在安装流水线的"校验后、首启前"窗口——与 rollback 共用 staging 思想。安全边界：备份默认端到端加密密钥挂 Google/OEM 账号，工作资料独立。

> **源码锚点 · 本章锚点**
> frameworks/native/cmds/installd/{Installd,CacheTracker,CrateManager}.cpp
frameworks/base/services/backup/（BackupManagerService 家族）
core/java/android/app/backup/（BackupAgent/BackupDataInput 输出侧 API）
split 加载：android/app/LoadedApk（书4 §1.4 注释① 同源）

第九章 补全：APK 签名体系、Intent 匹配与安装后布局

9.1 签名四代（包宇宙的信任根）

方案	位置	防什么	短板
v1（JAR 签名）	ZIP 内逐文件 META-INF/MANIFEST.MF 条目	篡改任意文件	校验慢、不支持未压缩对齐完整性、渠道包注入漏洞源头
v2（APK Signing Block）	ZIP 中央目录前的整块	**整个 ZIP 字节级**篡改（含中央目录）	7.0 以下不认
v3（+密钥轮换谱系）	Signing Block 内附 proof-of-rotation 链	老密钥签新密钥的**升级链伪造**	链验证复杂
v4（fs-verity 树）	单独 .idsig 文件	运行时按页增量校验（配合 dm-verity 思想）	仅安装期用

安装时 `ApkSignatureVerifier`（core/java/android/util/apk/，公共校验器）与 `pm/PackageSignatures.java`（服务端指纹记账）按"最高可用方案"验证；升级一致性校验（书6 §3.2）比对的正是这套指纹。**平台签名**（platform key 签的 priv-app）等同系统身份（书6 §4.2 特权白名单的另一条腿）；密钥库（Keymint/TEE，后续册）管的是运行时密钥，与 APK 签名是两个世界。渠道包的正确姿势：v2+ 的 ZIP 尾部注释区或 ID-value 对（signing block 的额外 ID），别再碰 v1 注入。

9.2 Intent 匹配算法：queryIntentActivities 的另一半

第四册每一步"问地址"的算法本体在 **IntentResolver**（`services/core/java/com/android/server/IntentResolver.java:448` queryIntent → `:440` buildResolveList → `:428` queryIntentFromList）：

text 匹配四关（任一不过即淘汰）

```
① action：Intent.action 命中 filter 任一 action（隐式 Intent 必须 action 非空）
② category：Intent 每个 category 都被 filter 包含（CATEGORY_DEFAULT 潜规则：
目标 filter 必须显式声明 DEFAULT 才能被隐式启动——"少写一行清单"的经典坑）
③ data：scheme→host→port→path（pathPattern/pathPrefix 通配）逐级收窄，
+ mimeType（type 与 scheme=content/file 的联动）
④ 优先级与排序：系统 App > 三方；matched filter 数多者胜；
EXPORTED=false 或权限不符者在返回前剔除
```

有了这张四关表，"为什么我的隐式 Intent 起不来"排查顺序自然成立：action→category(默认)→data/type→exported/权限——每一关都能用 `pm query-activities` 或 dumpsys package 的 filter 段验证。

9.3 安装后的物理布局（/data/app）

text /data/app/v~~<随机>/c~~<随机>/

```
├── base.apm→base.apk # 主体（split_*.apk 同级）
├── lib/<abi>/libxxx.so # 原生库（extractNativeLibs 决定是否解压）
└── oat/<abi>/base.odex|base.vdex # 编译产物（vdex=dex 缓存，odex=机器码，书6 §3.3）
```

`codePath/nativeLibraryDir` 记在 packages.xml（书6 §1.4 表）；首启的 LoadedApk 按它建类加载器与库路径。**v~~ 随机目录**防路径预测（书6 §3.1），oat 缺失则回落解释执行（书1 §8 三级策略的物理呈现）。

> **源码锚点 • 本章锚点**
> core/java/android/util/apk/ApkSignatureVerifier.java 校验 / pm/PackageSignatures.java 记账
services/core/.../IntentResolver.java:448/440/428
布局事实源：FileInstallArgs 落盘规则 + Settings.codePath

附录 A 观测实操

目标	命令	看点	
包全景	dumpsys package <pkg>	版本/签名/权限/用户态 /InstallSource	
组件解析	pm query-activities --components ... 或 dumpsys package intents	倒排索引视角	
安装过程	pm install -r -t + logcat -s PackageManager	五步管线逐步日志	
失败码	INSTALL_FAILED_*	对照 §3.2 校验段	
权限态	appops get <pkg> / pm list permissions -g	三态+即时开关	
dex 状态	pm art dump（13）/ dumpsys package	dexopt	artd 视角
清账	pm clear / pm uninstall --user 0	用户态与全机删除差异	

附录 B 快问快答（16 条）

packages.xml 在哪？——/data/system/packages.xml（Settings 账本）。 Vending（Play）怎么静默装？——INSTALL_PACKAGES 系统签名+会话。 升级系统 App 的apk 放哪生效？——/data/app 覆盖（同签名高版本）。 scanDirTracedLI 扫几个分区？——framework、system(priv+normal)、overlay、vendor、product、system_ext、data（:248/:320/:325 及后续）。

priv-app 特权两个条件？——分区位置+privapp-permissions 白名单。 RRO 是什么？——运行时资源覆盖层（overlay 分区扫描）。 Computer 与 PMS 关系？——不可变查询快照，写后原子换引用。 installStage 在哪个类？——InstallParams.java:320（13 聚合进 InstallPackageHelper）。

安装到一半断电？——staging 目录被弃，账未动，旧版无损。 PACKAGE_ADDED 广播为何有序？——REPLACED→ADDED 链条语义敏感（备份/MA 依赖顺序）。 13 的 dexopt 谁执行？——ART APEX 的 artd（installd 退守存储）。 checkPermission 三层？——声明/授予/AppOps。

targetSdk 为什么影响权限？——M/23、Q/29、R/30 行为分水岭按其分叉。 多用户装一份包？——本体一份，PackageUserState per-user 记账。 拨号器为何出厂就有通讯录权限？——DefaultPermissionGrantPolicy 角色授予。 下一步读什么？——书7 面试题集全主题串联六册。

附录 C 自检题

- 默画安装五步，标注每步的"原子性保障点"。
- 扫描顺序为什么 data 最后？签名校验失败时哪一步拒绝？
- Computer 快照换引用的瞬间，读者拿到的是新是旧？为什么没有中间态？
- 权限三层各自由谁撤销？举一条命令对应每层。

附录 D 动手实验室（半天量）

实验一"账本考古"：root 后打开 /data/system/packages.xml，找一个你装的 App，核对 §1.4 十个字段；再 dumpsys package 同名包对照两本账的同一事实。

实验二"管线直播"：`logcat -s PackageManager` 开着，`pm install -r` 一个测试包，把日志按 §3.2 五步分段认领；故意换签名装第二遍，收割 `INSTALL_FAILED_UPDATE_INCOMPATIBLE`。

实验三"快照体验"：安装的同时连续 `dumpsys package <pkg> | grep version`——观察读到的是旧版本直到广播后跳新（Computer 语义现场版）。

实验四"预装演练"：在可写设备（模拟器）把一个普通 apk 放进 /system/priv-app（配白名单 xml），重启验证：特权生效了吗？构建期校验在城市里已跳过——运行时如何提示缺失白名单（logcat 的 PrivilegedPermission 检索）。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1-5 题：见附录 B 快问快答（将其转化为"闭卷手写"练习）。

第二级：分析题（检验机制理解）

6. **追踪调用链**：画出本册核心链路的完整调用图（参考第一章/第N章图）。

7. **失败推演**：如果本册的核心机制在某一步失败，系统会怎样？

8. **对比分析**：本册机制与 Binder 十跳（书3）的交汇点在哪里？

第三级：设计题

9. **架构演进**：如果要重构本册核心机制，你会怎么改？

10. **排障方案**：用户报告本册领域的问题，设计分层定位方案。

术语表（按拼音排序）

术语	所在册	解释
（术语表：从附录 B 快问快答提取本册专属术语，按拼音排序）		

排障手册：PMS 与包管理问题定位

核心问题

1. dumpsys 定位状态

2. logcat 查看日志

3. perfetto/performance 分析性能

→ 深入：本册各章